```
Output:
Array
(
    [0] => 2
    [1] => 3
    [2] => 4
)
```

This works because we're accessing `$arr` directly. The iterator variable `$i` changes on each loop, but we're using `$arr` (which exists outside the `for` code block) for access to the elements, so our changes will stick. Doing something similar with `foreach` requires a little more effort, as we'll see very shortly. But first, a warning: be careful with `for` loops when adding elements, as this classic bug demonstrates:

```
$arr = ["Alice", "Bob"];
for ($i = 0; $i < count($arr); $i++) {
    print "Adding a Russell!" . PHP_EOL;
    $arr[] = "Russell";
}
print_r($arr);
```

```
Output:
Adding a Russell!
Adding a Russell!
Adding a Russell!
Adding a Russell!
... several thousand iterations  ...
PHP Fatal error:  Allowed memory size of 134217728 bytes exhausted...
```

The problem here is we're testing the size of the array on each iteration – `($i = 0; $i < count($arr); $i++)` – and the array is gaining an element on each loop. `$i` starts at `0`, and the return from `count($arr)` will always exceed it, so logically the loop can never end (we run out of memory first). You should always be careful with `count()` and `for` loops. Notice the previous code example uses the expression `for ($i = 0, $size = count($arr); $i < $size; ...` to initialize `$size` as the count of array elements at the start of the loop and compare `$i` to `$size` on each iteration. This not only makes the loop safe; it is also more efficient (otherwise we're calling the `count()` function on every iteration).